

UFPI - CSHNB

Curso de Bacharelado em Sistemas de Informação

Disciplina: Redes de Computadores II

Avaliação: Terceira Avaliação de Redes de Computadores II de 2026-1

Valor: 10,0 Pontos

Data de Entrega: 26/06/2026 (Trabalho Individual)

Tema: Evolução de Arquitetura: Servidor HTTP Simplificado sobre R-UDP/TCP e Resolução de Nomes Local (DNS)

1. Objetivo

Evoluir o sistema de transferência de arquivos desenvolvido na Segunda Avaliação (referenciada no arquivo "Primeira Avaliacao Redes 2 2026-1"), transformando o par Cliente/Servidor em um Miniservidor Web funcional (HTTP/1.1 simplificado) capaz de operar tanto sobre o protocolo TCP nativo quanto sobre a camada de confiabilidade R-UDP criada pelo aluno. Adicionalmente, será integrado um módulo de Resolução de Nomes (Mini-DNS) baseado em tabelas locais para simular o mapeamento de domínios no ambiente de testes.

2. Objetivos Específicos

- **Aproveitar e Adaptar:** Utilizar a estrutura de containers Docker, os scripts de controle de tráfego (tc) e a camada R-UDP (com controle de fluxo e de erros) desenvolvidos na avaliação anterior.
- **Camada de Aplicação (HTTP/1.1):** Implementar a análise de requisições GET legítimas para servir páginas HTML e arquivos estáticos simples. Agora os arquivos sairão do servidor WEB para o cliente WEB.
- **Resolução de Nomes (DNS):** Implementar um cliente e um servidor DNS minimalistas, operando exclusivamente via UDP na porta 53 (ou em porta customizada nos contêineres), para resolver o endereço IP do miniservidor web antes de iniciar a conexão HTTP.
- **Validação Cruzada:** Confrontar os tempos de resolução de DNS e o overhead do cabeçalho HTTP em diferentes cenários de rede utilizando o Wireshark/TCPDump.

3. O Projeto (Evolução)

O aluno deverá estender o projeto anterior criando a seguinte arquitetura funcional:

3.1. Módulo DNS Local (UDP nativo)

Antes de realizar qualquer requisição de arquivo, o Cliente não poderá utilizar endereços IP diretamente. Ele deve obrigatoriamente consultar um contêiner com o "Servidor DNS" (também implementado em Python pelo aluno) para obter o IP do Servidor Web (usar uma tabela simples em memória, ler um arquivo .txt ou outra forma).

- O servidor DNS responderá a consultas do tipo A (IPv4) com base em um arquivo de zona estático (hosts.txt).
- Toda requisição e resposta DNS devem seguir o formato simplificado de cabeçalho do protocolo (contendo apenas os campos ID, Name e IP).

3.2. Módulo HTTP/1.1 (Sobre TCP e R-UDP)

Uma vez resolvido o IP via DNS, o Cliente fará uma requisição HTTP GET ao Servidor. O Servidor deve processar e responder com os cabeçalhos padrão: HTTP/1.1 200 OK (ou 404 Not Found), Content-Type, Content-Length e manter o campo personalizado X-Custom-Auth (Matrícula + Nome) introduzido na avaliação passada.

- O sistema deve permitir alternar dinamicamente entre o transporte HTTP via TCP nativo e via R-UDP.

3.3. Ambiente de Teste e Simulação

Serão utilizados os mesmos containers Docker, mantendo as simulações obrigatórias com tc qdisc focando nos cenários de estresse da prova anterior para avaliar o impacto na navegação Web simulada (mesmo valores que no primeiro trabalho):

- **Cenário A:** 0% de perda / 10 ms de delay.
- **Cenário B:** 5% de perda / 50 ms de delay.
- **Cenário C:** 10% de perda / 100 ms de delay.

4. Critérios de Avaliação (10 Pontos)

A nota final será computada com base na entrega e funcionamento dos seguintes módulos e artefatos:

Critério	Descrição	Pontos
Integração e Reuso	Reuso bem-sucedido da camada R-UDP e do ambiente multi-container do Docker, adaptado para incluir o nó DNS.	1.5
Módulo DNS Local	Implementação correta do servidor/cliente DNS simplificado sobre UDP e resolução por arquivo local.	2.5
Miniservidor HTTP/1.1	Suporte a requisições GET, tratamento de erro 404, envio correto de cabeçalhos básicos e o header X-Custom-Auth.	2.5
Validação Wireshark & Gráficos	Capturas .pcap comprovando a sequência lógica (DNS -> HTTP) e gráficos comparativos de tempo total de carregamento (TCP vs R-UDP).	1.5
Relatório (SBC) & Respostas	Discussão técnica dos resultados obtidos e respostas fundamentadas às perguntas obrigatórias.	1.0
Vídeo Demonstrativo	Explicação em vídeo (máx. 15 min) mostrando o funcionamento integrado com perda de pacotes ativa.	1.0

5. Entrega Esperada

1. **Código no GitHub:** Sockets atualizados (Cliente, Servidor Web, Servidor DNS), Dockerfile/Docker-compose e script de geração de gráficos em Python.
2. **Arquivos de Captura:** Logs .pcap que contenham o fluxo completo correlacionando a atividade de rede.
3. **Relatório em PDF:** Modelo SBC atualizado com as novas análises comparativas e dados estatísticos.
4. **Vídeo no YouTube:** Demonstração prática do sistema em funcionamento.

6. Perguntas Obrigatórias no Relatório

1. Como a perda de pacotes simulada no canal afetou o tempo de resolução DNS (que usa UDP nativo sem retransmissão na camada de transporte) em comparação com o download da página via HTTP (R-UDP/TCP)? O cliente DNS precisou implementar algum timeout na aplicação?
2. Qual foi o impacto visual (na análise de gráficos) e métrico do overhead dos cabeçalhos HTTP adicionados nesta avaliação, quando comparado ao protocolo de aplicação puramente textual e customizado da Segunda Avaliação?
3. Ao inspecionar o Wireshark, o fluxo de pacotes seguiu estritamente a ordem esperada da arquitetura da Internet? Demonstre através de prints do fluxo de tempo do Wireshark a transição exata entre o encerramento da query DNS e o início do handshake TCP / primeira transmissão R-UDP.

7. Gráfico Esperados:

- Taxa de transferência com DNS nos 3 cenários usando o TCP e o R-DUP
- Usar arquivo de tamanho diferente ([p.ex.](#): 100kB, 1MB e 10MB).
- Executar cada cenário pelo menos 10 vezes para gerar médias, desvios-padrão, mínimos e máximos das taxas de transferência e de erro.

Bom Trabalho!